Lab 7: Fuzzing for Input Validation Bugs

Name: ARABHIBHAT

SRN: PES1UG24CS927

Application: restaurant_bill_parser.py (repo L6_L7_58_restaurant_bill, LAB7)

Tools: Python, Hypothesis, pytest

1. Approach

The application has three functions that read text typed by a user: clean_dish_name, parse_item_prices and split_amount. The original code assumed every input was well formed. Each function has three Hypothesis tests. The no_crash test (the starter To-Do) feeds random Unicode text or None and fails if any exception is raised. The bad_values test builds text from the characters these inputs really use (digits, commas, '/', '-', spaces and the *, #, ~ markers), so Hypothesis quickly reaches cases like '1200/0', '250,,180' and '-5'; the result must be None or a valid value. The valid_input test builds well-formed input from real numbers and checks that the answer is still correct.

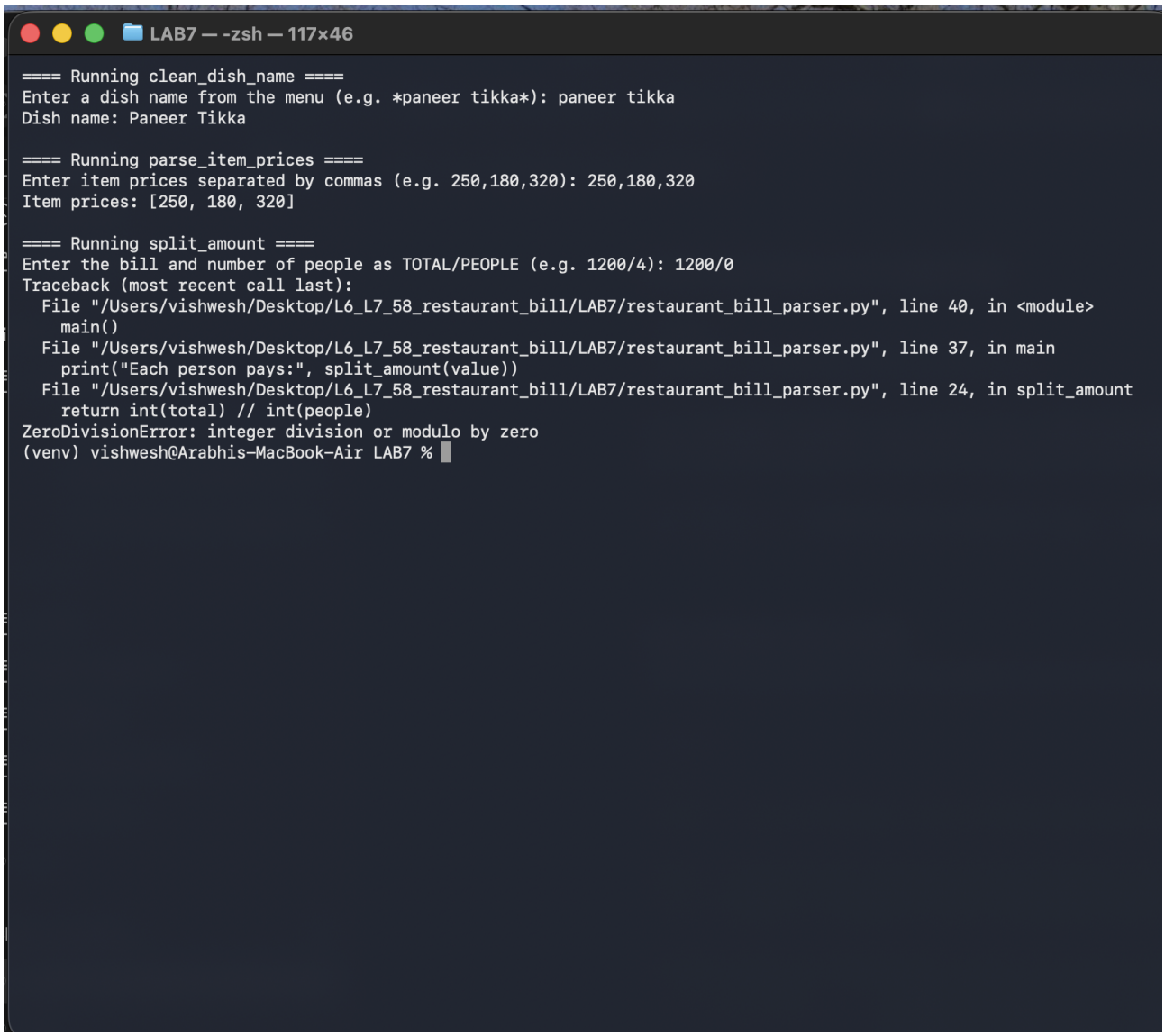
2. Bugs found and fixes

Function	Fuzzed input	Original behaviour	After fix
clean_dish_name	None	AttributeError (no .strip)	Returns None
clean_dish_name	'tikka *'	'Tikka ' (space left after the marker)	'Tikka' (strip after removing markers)
clean_dish_name	'', ' ', '***'	'' (empty name accepted)	Returns None
parse_item_prices	None	AttributeError (no .split)	Returns None
parse_item_prices	'', '250,,180', '250,180,'	ValueError: int('')	Blank entries skipped
parse_item_prices	'abc', '1.5', '1_000', superscript 2	ValueError or accepted	Returns None
parse_item_prices	'-5,10'	[-5, 10] (negative price)	Returns None
split_amount	None	AttributeError (no .split)	Returns None
split_amount	'1200', '1/2/3'	ValueError (unpacking)	Returns None
split_amount	'abc/4'	ValueError: int('abc')	Returns None
split_amount	'1200/0'	ZeroDivisionError	Returns None
split_amount	'-1200/4'	-300 (negative share)	Returns None

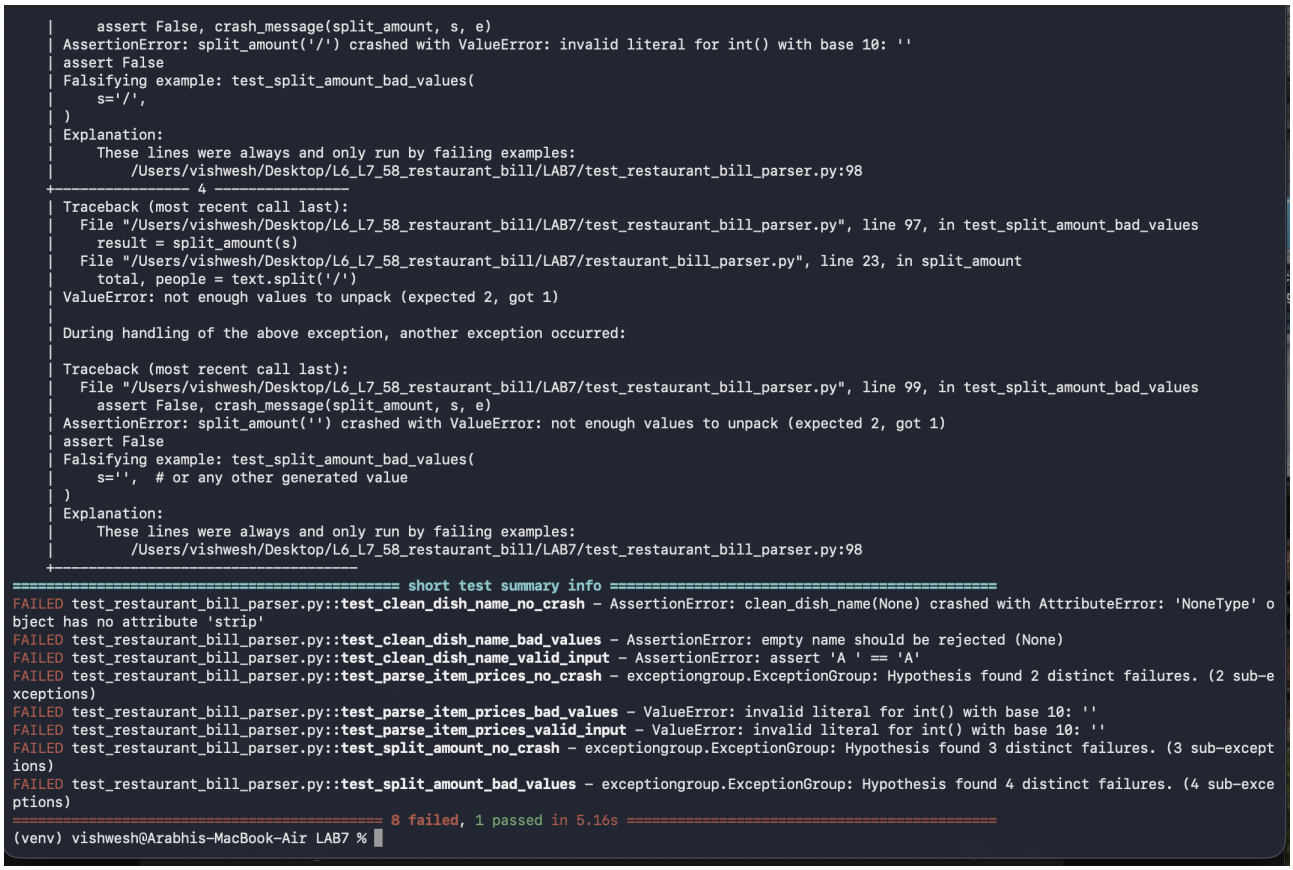
After the fix no function raises an exception. Each one returns a clean result for valid input and None for invalid input, and main() prints a clear message instead of crashing. Numbers must be written with the digits 0-9 only (str.isdigit() alone is not enough, because it is also True for a superscript 2 that int() cannot convert). In clean_dish_name the markers are now removed before stripping spaces.

3. Screenshots

Step 2: Running the original application (python restaurant_bill_parser.py)



Step 4: Fuzz tests failing on the original code (pytest test_restaurant_bill_parser.py)



Step 5: Running the fixed application (python restaurant_bill_parser.py)

```
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % ls *.png
1_original_run.png      2_tests_failing.png
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % pbpaste>restaurant_bill_parser.py
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % head -2 restaurant_bill_parser.py
"""
Lab 7 - Fuzz Testing: restaurant_bill_parser.py (FIXED)
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % clear

(venv) vishwesh@Arabhis-MacBook-Air LAB7 % python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): paneer tikka
Dish name: Paneer Tikka

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,180,320
Item prices: [250, 180, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/4
Each person pays: 300
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): paneer tikka
Dish name: Paneer Tikka

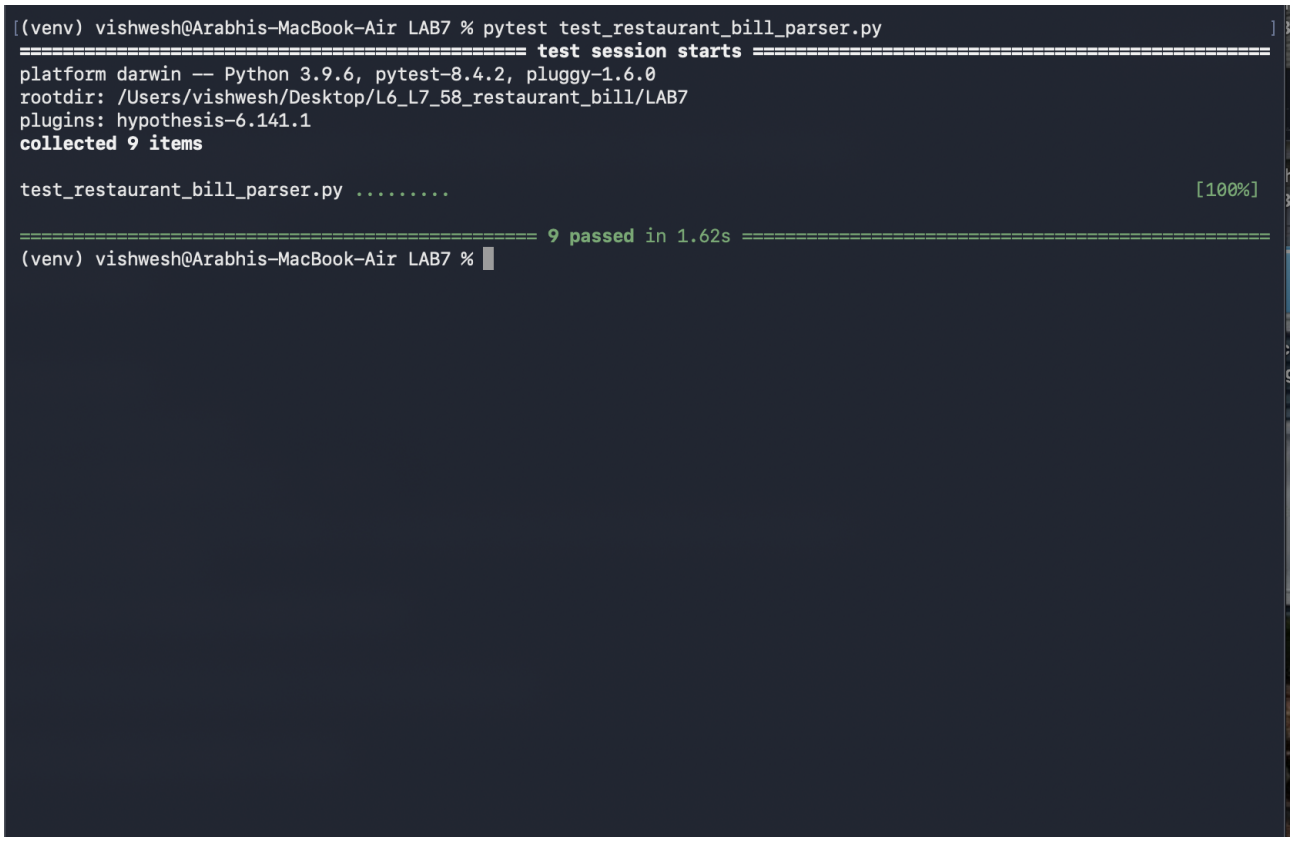
==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,180,320
Item prices: [250, 180, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/0
Each person pays: Invalid input - use TOTAL/PEOPLE with whole numbers and PEOPLE > 0
(venv) vishwesh@Arabhis-MacBook-Air LAB7 % python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): ***
Dish name: Invalid dish name - please enter a name

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,abc,-5
Item prices: Invalid prices - use whole numbers separated by commas

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/0
Each person pays: Invalid input - use TOTAL/PEOPLE with whole numbers and PEOPLE > 0
(venv) vishwesh@Arabhis-MacBook-Air LAB7 %
```

Step 6: All fuzz tests passing on the fixed code (pytest test_restaurant_bill_parser.py)



4. Conclusion

Fuzz testing with Hypothesis found crashes and silent wrong answers that the starter unit test did not, because it only used the inputs we expected. After adding validation, all nine Hypothesis tests pass.

Appendix A: restaurant_bill_parser.py

```
"""
Lab 7 - Fuzz Testing: restaurant_bill_parser.py (FIXED)

The original code assumed every input was well formed. Fuzzing with
Hypothesis showed it crashing (AttributeError, ValueError,
ZeroDivisionError) or returning wrong values (negative prices,
negative shares, names with stray spaces).

Rule after the fix: no function ever raises an exception. Each one
returns a clean result for valid input and None for invalid input,
so the caller can tell the user what went wrong.
"""

MARKERS = ['*', '#', '~']


def _to_whole_number(text):
    """Turn text like ' 250 ' into 250. Return None if it is not a
    whole number written with the digits 0-9.

    isascii() is needed because isdigit() is also True for characters
    like '²' that int() cannot convert.
    """
    text = text.strip()
    if not (text.isascii() and text.isdigit()):
        return None             # '', '-5', '+5', '1.5', '1_000', 'abc', '²'
    try:
        return int(text)
    except ValueError:          # more than 4300 digits (Python's int limit)
        return None


def clean_dish_name(name):
    """
    Removes the chef's-special markers (*, #, ~) and title-cases the name.
    Returns None if name is not a string or nothing is left after cleaning.
    """
    if not isinstance(name, str):           # FIX: None crashed with AttributeError
        return None
    for ch in MARKERS:
        name = name.replace(ch, '')
    name = name.strip()                     # FIX: strip AFTER removing markers,
                                            # so 'tikka *' gives 'Tikka' not 'Tikka '
    if not name:                            # FIX: '', '   ', '***' gave ''
        return None
    return name.title()


def parse_item_prices(text):
    """
    Parses comma-separated prices into a list of ints.
    Blank entries (e.g. '250,,180' or a trailing comma) are skipped.
    Returns None if text is not a string or any price is not a whole number.
    """
    if not isinstance(text, str):           # FIX: None crashed with AttributeError
        return None
    prices = []
    for part in text.split(','):
        if not part.strip():                # FIX: '' and '250,,180' crashed
            continue
        price = _to_whole_number(part)
        if price is None:                   # FIX: 'abc', '1.5' crashed;
            return None                     # '-5' was accepted as a price
        prices.append(price)
    return prices


def split_amount(text):
    """
    Splits 'TOTAL/PEOPLE' into the amount each person pays.
    Returns None if text is not in that form, either value is not a
    whole number, or people is zero.
    """
    if not isinstance(text, str):           # FIX: None crashed with AttributeError
        return None
    parts = text.split('/')
    if len(parts) != 2:                     # FIX: '1200' and '1/2/3' crashed
        return None
```

```
    total = _to_whole_number(parts[0])      # FIX: 'abc/4' crashed;
    people = _to_whole_number(parts[1])     # '-1200/4' gave a negative share
    if total is None or people is None:
        return None
    if people == 0:                         # FIX: '1200/0' crashed with
        return None                         # ZeroDivisionError
    return total // people


def main():
    print("==== Running clean_dish_name ====")
    value = input("Enter a dish name from the menu (e.g. *paneer tikka*): ")
    result = clean_dish_name(value)
    print("Dish name:", result if result is not None
          else "Invalid dish name - please enter a name")

    print("\n==== Running parse_item_prices ====")
    value = input("Enter item prices separated by commas (e.g. 250,180,320): ")
    result = parse_item_prices(value)
    print("Item prices:", result if result is not None
          else "Invalid prices - use whole numbers separated by commas")

    print("\n==== Running split_amount ====")
    value = input("Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): ")
    result = split_amount(value)
    print("Each person pays:", result if result is not None
          else "Invalid input - use TOTAL/PEOPLE with whole numbers and PEOPLE > 0")


if __name__ == "__main__":
    main()
```

Appendix B: test_restaurant_bill_parser.py

```
"""
Lab 7 - Fuzz Testing: test_restaurant_bill_parser.py

Run:  pytest test_restaurant_bill_parser.py
 or:  python -m pytest test_restaurant_bill_parser.py

Three kinds of Hypothesis tests for each function:
  1. no_crash    - random Unicode text or None must never raise an
                   exception (the three starter To-Do tests).
  2. bad_values  - text built from the characters these inputs really
                   use (digits, commas, '/', '-', spaces, markers), so
                   Hypothesis quickly reaches cases like '1200/0',
                   '250,,180' and '-5'. The result must be None or a
                   valid value - never a negative price or share.
  3. valid_input - well-formed input built from real numbers must
                   still give the right answer after the fix.
"""
from hypothesis import given, settings, strategies as st

from restaurant_bill_parser import clean_dish_name, parse_item_prices, split_amount

settings.register_profile("lab7", max_examples=300, deadline=None)
settings.load_profile("lab7")

MARKERS = "*#~"
names_text = st.text(alphabet=" abcXYZ" + MARKERS + "\t")
prices_text = st.text(alphabet="0123456789,- .a")
split_text = st.text(alphabet="0123456789/- .a")


def crash_message(func, s, e):
    return f"{func.__name__}({s!r}) crashed with {type(e).__name__}: {e}"


# ------------------------------------------------------------ clean_dish_name
@given(st.text() | st.none())
def test_clean_dish_name_no_crash(s):
    try:
        clean_dish_name(s)
    except Exception as e:
        assert False, crash_message(clean_dish_name, s, e)


@given(names_text)
def test_clean_dish_name_bad_values(s):
    result = clean_dish_name(s)
    if result is not None:
        assert result != "", "empty name should be rejected (None)"
        assert result == result.strip(), f"stray spaces left in {result!r}"
        assert not any(m in result for m in MARKERS), f"marker left in {result!r}"


@given(st.text(alphabet="abcdefg ", min_size=1).filter(str.strip),
       st.sampled_from(["*", "#", "~", "**", "~#"]))
def test_clean_dish_name_valid_input(name, marker):
    expected = name.strip().title()
    assert clean_dish_name(marker + name + marker) == expected
    assert clean_dish_name(name) == expected


# ------------------------------------------------------------ parse_item_prices
@given(st.text() | st.none())
def test_parse_item_prices_no_crash(s):
    try:
        parse_item_prices(s)
    except Exception as e:
        assert False, crash_message(parse_item_prices, s, e)


@given(prices_text)
def test_parse_item_prices_bad_values(s):
    result = parse_item_prices(s)
    if result is not None:
        assert isinstance(result, list)
        assert all(isinstance(p, int) and p >= 0 for p in result), \
            f"invalid price in {result!r} from {s!r}"
@given(st.lists(st.integers(min_value=0, max_value=100000)),
       st.sampled_from([",", ", ", " , "]))
```

```
def test_parse_item_prices_valid_input(prices, sep):
    assert parse_item_prices(sep.join(str(p) for p in prices)) == prices


# ------------------------------------------------------------ split_amount
@given(st.text() | st.none())
def test_split_amount_no_crash(s):
    try:
        split_amount(s)
    except Exception as e:
        assert False, crash_message(split_amount, s, e)


@given(split_text)
def test_split_amount_bad_values(s):
    try:
        result = split_amount(s)
    except Exception as e:
        assert False, crash_message(split_amount, s, e)
    if result is not None:
        assert isinstance(result, int) and result >= 0, \
            f"split_amount({s!r}) gave {result!r}"


@given(st.integers(min_value=0, max_value=10**7),
       st.integers(min_value=1, max_value=100))
def test_split_amount_valid_input(total, people):
    assert split_amount(f"{total}/{people}") == total // people
    assert split_amount(f" {total} / {people} ") == total // people
```